CHAPTER 6

Persisting Data with Ecto

Welcome to Part II! We're going to change things up a bit in the second part of this book. The first part was focused on learning the language and getting comfortable with Elixir, but the second part is focused on learning tools and building a real application. Throughout these next chapters, you'll build an SMS app that sends real text messages. We'll implement new features in each chapter until there's a complete product.

It's important to build things from scratch. That's how you learn the fine details. So, you'll start with `mix phx.new` and write all of the code yourself. However, some things like CSS styles and HTML are provided in a way that you can easily copy into your application. This is because they are often long code listings that are tedious to type by hand. And don't worry—you can start with a provided code package at the beginning of each chapter if that's your preference.

Data is the foundation of all applications. We ingest data from users or APIs, we manipulate data with queries and processing, and we show users their data so they can act on it. Data persistence and access is a pretty big deal!

Ecto is Elixir's answer for everything data and database-related. Ecto differs significantly from ActiveRecord, but it still feels familiar due to a design that's focused on developer productivity.

We'll start the chapter by going over the project we're going to build. Then we'll cover Ecto's design and the philosophy it takes. Of course, we'll compare it to ActiveRecord along the way. You'll write an Ecto schema to represent SMS messages and then use Ecto changesets to persist data into the database. Finally, you'll learn several different ways to query data and how to best expose data functions to your application. We'll wrap all of our functions into a context that will be used later in the project.